Spring Data JPA Cheat Sheet

Setup & Dependencies

Maven Dependency

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
</dependency>
```

Application Properties

```properties
# Database Configuration
spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.datasource.username=root
spring.datasource.password=password
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# JPA Configuration
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.database-platform=org.hibernate.dialect.MySQL8Dialect
```

Entity Annotations

Basic Entity

```java
@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "email", unique = true, nullable = false)
    private String email;

    @Column(length = 100)
    private String name;

    @Temporal(TemporalType.TIMESTAMP)
    @CreationTimestamp
    private Date createdAt;

    @UpdateTimestamp
    private LocalDateTime updatedAt;

    // Constructors, getters, setters
}
```

Common Annotations

- `@Entity` - Marks class as JPA entity
- `@Table(name = "table_name")` - Specifies table name
- `@Id` - Primary key
- `@GeneratedValue` - Auto-generated values
- `@Column` - Column mapping
- `@Temporal` - Date/time mapping
- `@Enumerated` - Enum mapping
- `@Lob` - Large objects (BLOB/CLOB)
- `@Transient` - Exclude from persistence

Generation Strategies

java

```java
@GeneratedValue(strategy = GenerationType.IDENTITY)  // Auto-increment
@GeneratedValue(strategy = GenerationType.SEQUENCE)  // Sequence
@GeneratedValue(strategy = GenerationType.TABLE)     // Table generator
@GeneratedValue(strategy = GenerationType.UUID)      // UUID
```

Relationships

One-to-Many

java

```java
@Entity
public class Author {
    @OneToMany(mappedBy = "author", cascade = CascadeType.ALL, fetch = FetchType.LAZY)
    private List<Book> books = new ArrayList<>();
}

@Entity
public class Book {
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "author_id")
    private Author author;
}
```

Many-to-Many

java

```java
@Entity
public class Student {
    @ManyToMany
    @JoinTable(
        name = "student_course",
        joinColumns = @JoinColumn(name = "student_id"),
        inverseJoinColumns = @JoinColumn(name = "course_id")
    )
    private Set<Course> courses = new HashSet<>();
}
```

One-to-One

```java
@Entity
public class User {
    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(name = "profile_id")
    private UserProfile profile;
}
```

Repository Interface

Basic Repository

```java
public interface UserRepository extends JpaRepository<User, Long> {
    // Built-in methods available:
    // save(), findById(), findAll(), deleteById(), count(), etc.
}
```

Repository Hierarchy

- `Repository<T, ID>` - Base interface
- `CrudRepository<T, ID>` - Basic CRUD operations
- `PagingAndSortingRepository<T, ID>` - Pagination & sorting
- `JpaRepository<T, ID>` - JPA specific (most commonly used)

Query Methods

Method Name Queries

java

```java
public interface UserRepository extends JpaRepository<User, Long> {
    // Find by field
    List<User> findByEmail(String email);
    List<User> findByName(String name);

    // Multiple conditions
    List<User> findByNameAndEmail(String name, String email);
    List<User> findByNameOrEmail(String name, String email);

    // Comparison operators
    List<User> findByAgeGreaterThan(int age);
    List<User> findByAgeLessThanEqual(int age);
    List<User> findByAgeBetween(int start, int end);

    // String operations
    List<User> findByNameContaining(String name);
    List<User> findByNameStartingWith(String prefix);
    List<User> findByNameEndingWith(String suffix);
    List<User> findByNameIgnoreCase(String name);

    // Null checks
    List<User> findByEmailIsNull();
    List<User> findByEmailIsNotNull();

    // Collections
    List<User> findByRolesIn(Collection<String> roles);
    List<User> findByRolesNotIn(Collection<String> roles);

    // Boolean
    List<User> findByActiveTrue();
    List<User> findByActiveFalse();

    // Ordering
    List<User> findByNameOrderByCreatedAtDesc(String name);

    // Limiting results
    User findFirstByEmail(String email);
    List<User> findTop5ByOrderByCreatedAtDesc();

    // Existence checks
    boolean existsByEmail(String email);

    // Counting
```

```java
    long countByActive(boolean active);

    // Deletion
    void deleteByEmail(String email);
    long removeByActive(boolean active);
}
```

Custom Queries

@Query Annotation

```java
public interface UserRepository extends JpaRepository<User, Long> {

    // JPQL Query
    @Query("SELECT u FROM User u WHERE u.email = ?1")
    User findByEmailJPQL(String email);

    // Named parameters
    @Query("SELECT u FROM User u WHERE u.name = :name AND u.active = :active")
    List<User> findByNameAndActive(@Param("name") String name, @Param("active") boolean active)

    // Native SQL Query
    @Query(value = "SELECT * FROM users WHERE email = ?1", nativeQuery = true)
    User findByEmailNative(String email);

    // Modifying queries
    @Modifying
    @Transactional
    @Query("UPDATE User u SET u.active = :active WHERE u.id = :id")
    int updateUserActive(@Param("id") Long id, @Param("active") boolean active);

    // Delete query
    @Modifying
    @Transactional
    @Query("DELETE FROM User u WHERE u.active = false")
    int deleteInactiveUsers();
}
```

Complex Queries

```java
// Join queries
@Query("SELECT u FROM User u JOIN u.roles r WHERE r.name = :roleName")
List<User> findByRoleName(@Param("roleName") String roleName);

// Aggregation
@Query("SELECT COUNT(u) FROM User u WHERE u.active = true")
long countActiveUsers();

// Group by
@Query("SELECT u.department, COUNT(u) FROM User u GROUP BY u.department")
List<Object[]> countUsersByDepartment();

// Subqueries
@Query("SELECT u FROM User u WHERE u.id IN (SELECT o.userId FROM Order o WHERE o.total > :amour
List<User> findUsersWithOrdersAbove(@Param("amount") BigDecimal amount);
```

Pagination and Sorting

Using Pageable

```java
public interface UserRepository extends JpaRepository<User, Long> {
    Page<User> findByActive(boolean active, Pageable pageable);
    List<User> findByNameContaining(String name, Sort sort);
}

// Usage in Service/Controller
@Service
public class UserService {

    public Page<User> getUsers(int page, int size) {
        Pageable pageable = PageRequest.of(page, size, Sort.by("name").ascending());
        return userRepository.findAll(pageable);
    }

    public Page<User> getActiveUsers(int page, int size, String sortBy) {
        Pageable pageable = PageRequest.of(page, size, Sort.by(sortBy).descending());
        return userRepository.findByActive(true, pageable);
    }
}
```

Sort Examples

```java
// Single field
Sort sort = Sort.by("name");
Sort sortDesc = Sort.by("name").descending();

// Multiple fields
Sort multiSort = Sort.by("name").and(Sort.by("createdAt").descending());

// Using Sort.Order
Sort complexSort = Sort.by(
    Sort.Order.asc("name"),
    Sort.Order.desc("createdAt")
);
```

Specifications (Criteria API)

Setup

```java
public interface UserRepository extends JpaRepository<User, Long>, JpaSpecificationExecutor<Use
}
```

Specification Examples

java

```java
public class UserSpecifications {

    public static Specification<User> hasName(String name) {
        return (root, query, cb) ->
            name == null ? cb.conjunction() : cb.equal(root.get("name"), name);
    }

    public static Specification<User> isActive(boolean active) {
        return (root, query, cb) -> cb.equal(root.get("active"), active);
    }

    public static Specification<User> emailContains(String email) {
        return (root, query, cb) ->
            cb.like(cb.lower(root.get("email")), "%" + email.toLowerCase() + "%");
    }

    public static Specification<User> createdAfter(LocalDateTime date) {
        return (root, query, cb) -> cb.greaterThan(root.get("createdAt"), date);
    }
}

// Usage
@Service
public class UserService {

    public List<User> findUsers(String name, String email, boolean active) {
        Specification<User> spec = Specification.where(null);

        if (name != null) {
            spec = spec.and(UserSpecifications.hasName(name));
        }
        if (email != null) {
            spec = spec.and(UserSpecifications.emailContains(email));
        }
        spec = spec.and(UserSpecifications.isActive(active));

        return userRepository.findAll(spec);
    }
}
```

Projections

Interface Projections

```java
public interface UserSummary {
    String getName();
    String getEmail();

    // Nested projection
    AddressSummary getAddress();

    interface AddressSummary {
        String getCity();
        String getCountry();
    }
}

// Repository method
List<UserSummary> findByActive(boolean active);
```

Class Projections (DTOs)

```java
public class UserDTO {
    private String name;
    private String email;

    public UserDTO(String name, String email) {
        this.name = name;
        this.email = email;
    }
    // getters, setters
}

// Repository method
@Query("SELECT new com.example.UserDTO(u.name, u.email) FROM User u WHERE u.active = true")
List<UserDTO> findActiveUserDTOs();
```

Transaction Management

Basic Transaction

```java
@Service
@Transactional
public class UserService {

    @Transactional(readOnly = true)
    public User findById(Long id) {
        return userRepository.findById(id).orElse(null);
    }

    @Transactional
    public User save(User user) {
        return userRepository.save(user);
    }

    @Transactional(rollbackFor = Exception.class)
    public void complexOperation() {
        // Multiple database operations
    }
}
```

Auditing

Setup

```java
@Configuration
@EnableJpaAuditing
public class JpaConfig {

    @Bean
    public AuditorAware<String> auditorProvider() {
        return () -> Optional.of("system"); // Or get from security context
    }
}
```

Auditable Entity

java

```java
@Entity
@EntityListeners(AuditingEntityListener.class)
public class User {

    @CreatedDate
    private LocalDateTime createdDate;

    @LastModifiedDate
    private LocalDateTime lastModifiedDate;

    @CreatedBy
    private String createdBy;

    @LastModifiedBy
    private String lastModifiedBy;

    @Version
    private Long version; // Optimistic locking
}
```

Common Patterns

Service Layer

java

```java
@Service
@Transactional
public class UserService {

    @Autowired
    private UserRepository userRepository;

    @Transactional(readOnly = true)
    public Optional<User> findById(Long id) {
        return userRepository.findById(id);
    }

    public User save(User user) {
        return userRepository.save(user);
    }

    public void deleteById(Long id) {
        userRepository.deleteById(id);
    }

    @Transactional(readOnly = true)
    public Page<User> findAll(Pageable pageable) {
        return userRepository.findAll(pageable);
    }
}
```

Exception Handling

```java
@Service
public class UserService {

    public User findByIdOrThrow(Long id) {
        return userRepository.findById(id)
            .orElseThrow(() -> new EntityNotFoundException("User not found with id: " + id));
    }

    public User findByEmailOrThrow(String email) {
        return userRepository.findByEmail(email)
            .orElseThrow(() -> new EntityNotFoundException("User not found with email: " + emai
    }
}
```

Performance Tips

Fetch Types

```java
@OneToMany(fetch = FetchType.LAZY)    // Default for collections
@ManyToOne(fetch = FetchType.EAGER)   // Default for single entities

// Use @EntityGraph to override fetch type
@EntityGraph(attributePaths = {"roles", "profile"})
Optional<User> findById(Long id);
```

Batch Operations

```java
@Repository
public class UserRepositoryImpl {

    @PersistenceContext
    private EntityManager entityManager;

    @Transactional
    public void batchInsert(List<User> users) {
        for (int i = 0; i < users.size(); i++) {
            entityManager.persist(users.get(i));
            if (i % 50 == 0) { // Batch size
                entityManager.flush();
                entityManager.clear();
            }
        }
    }
}
```

Query Optimization

```java
// Use projections for read-only operations
@Query("SELECT u.name, u.email FROM User u WHERE u.active = true")
List<Object[]> findActiveUserNames();

// Use EXISTS for better performance
@Query("SELECT u FROM User u WHERE EXISTS (SELECT 1 FROM u.orders o WHERE o.status = 'ACTIVE')"
List<User> findUsersWithActiveOrders();
```

Testing

Repository Testing

java

```java
@DataJpaTest
class UserRepositoryTest {

    @Autowired
    private TestEntityManager entityManager;

    @Autowired
    private UserRepository userRepository;

    @Test
    void testFindByEmail() {
        // Given
        User user = new User("test@example.com", "Test User");
        entityManager.persistAndFlush(user);

        // When
        Optional<User> found = userRepository.findByEmail("test@example.com");

        // Then
        assertThat(found).isPresent();
        assertThat(found.get().getName()).isEqualTo("Test User");
    }
}
```